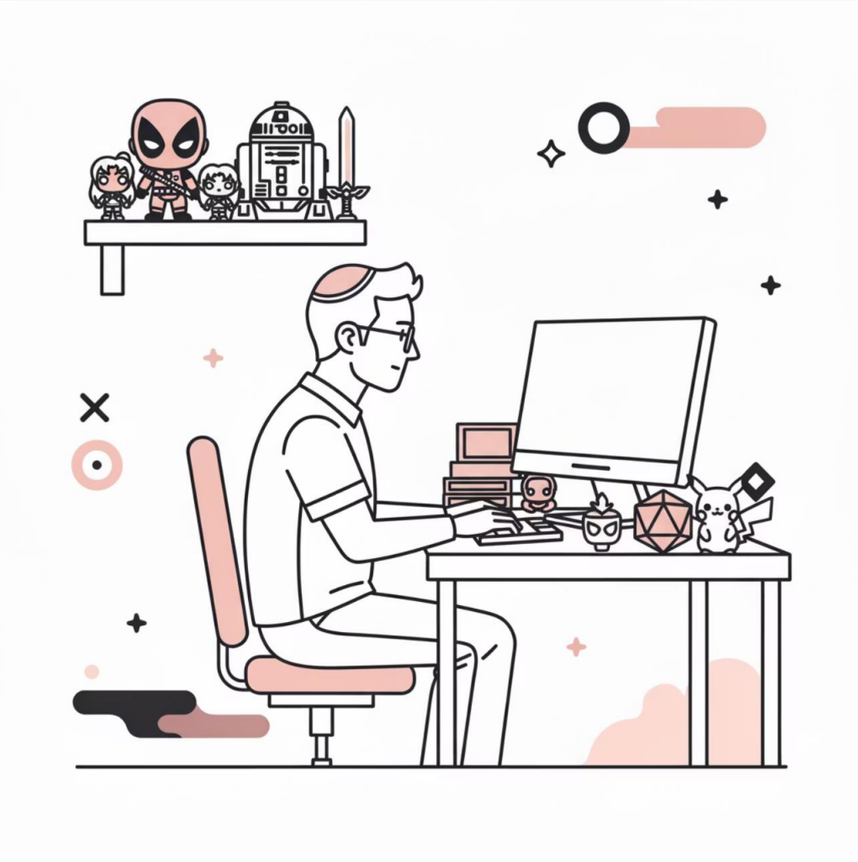
RTFM 2.0: Read the Feedback, Make It Better

A new approach to documentation and user feedback

tufin



About Me



Yehoshua Paul

Technical Communicator, Tufin

Best known for emptying the feedback inbox

(and other unrelated feats)

RTFM (Version 1.0)

Documentation exists

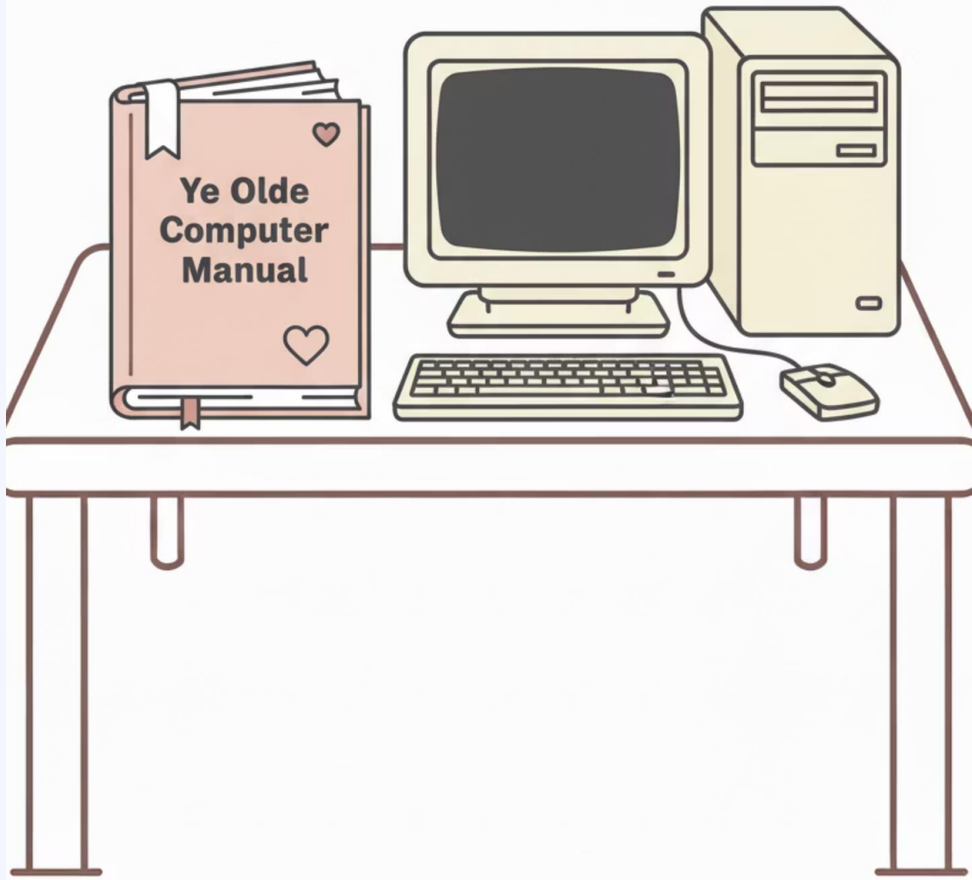
Someone wrote it. It's published. Done.

User responsibility

Read the manual. Figure it out.

Dismissal

"It's all in the user guide" — conversation over.



But Then...

Users engaged

Users open the docs, follow steps, try to understand

Users got confused

Something doesn't work. Context is missing.

Users asked questions

emails, tickets, Slack messages flood in.
And sometimes bugs.

Now what?

The responsibility shifts back to writers



RTFM 2.0

Users read

They engage with documentation

Users respond

Feedback flows back

Writers listen

And make it better



What This Talk Covers

01

What and Why

What feedback shows and why it matters

02

Reading the Feedback

Understanding the user

03

Who Gives the Feedback

Product managers, support, R&D, end-users

04

Feedback Channels

Friction vs. Quality

05

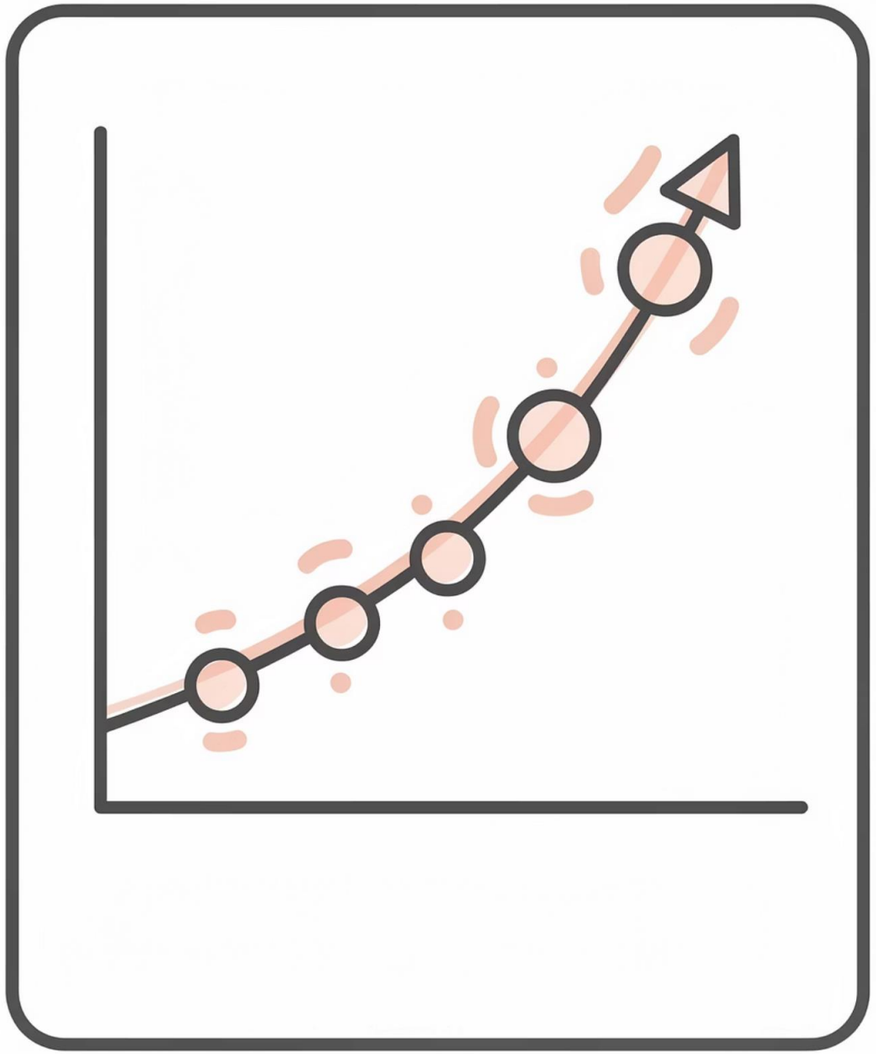
Managing Feedback

From submission to publication

06

Tracking Feedback

What do the patterns teach?



CHAPTER 1

What and Why

What feedback highlights

Gaps that need to be addressed

What feedback isn’t

An unnecessary distraction

Why it matters

There is a lot of it

What Feedback Is Not



What Documentation Feedback Highlights



Clarity gaps

Users can't find what they need or understand what they found



Broken assumptions

Writers assumed context users don't have



Missing context

Steps work in theory, fail in practice



Feedback Is an Operational Signal

Clarity

Are users understanding?

Usage

What's actually being read?

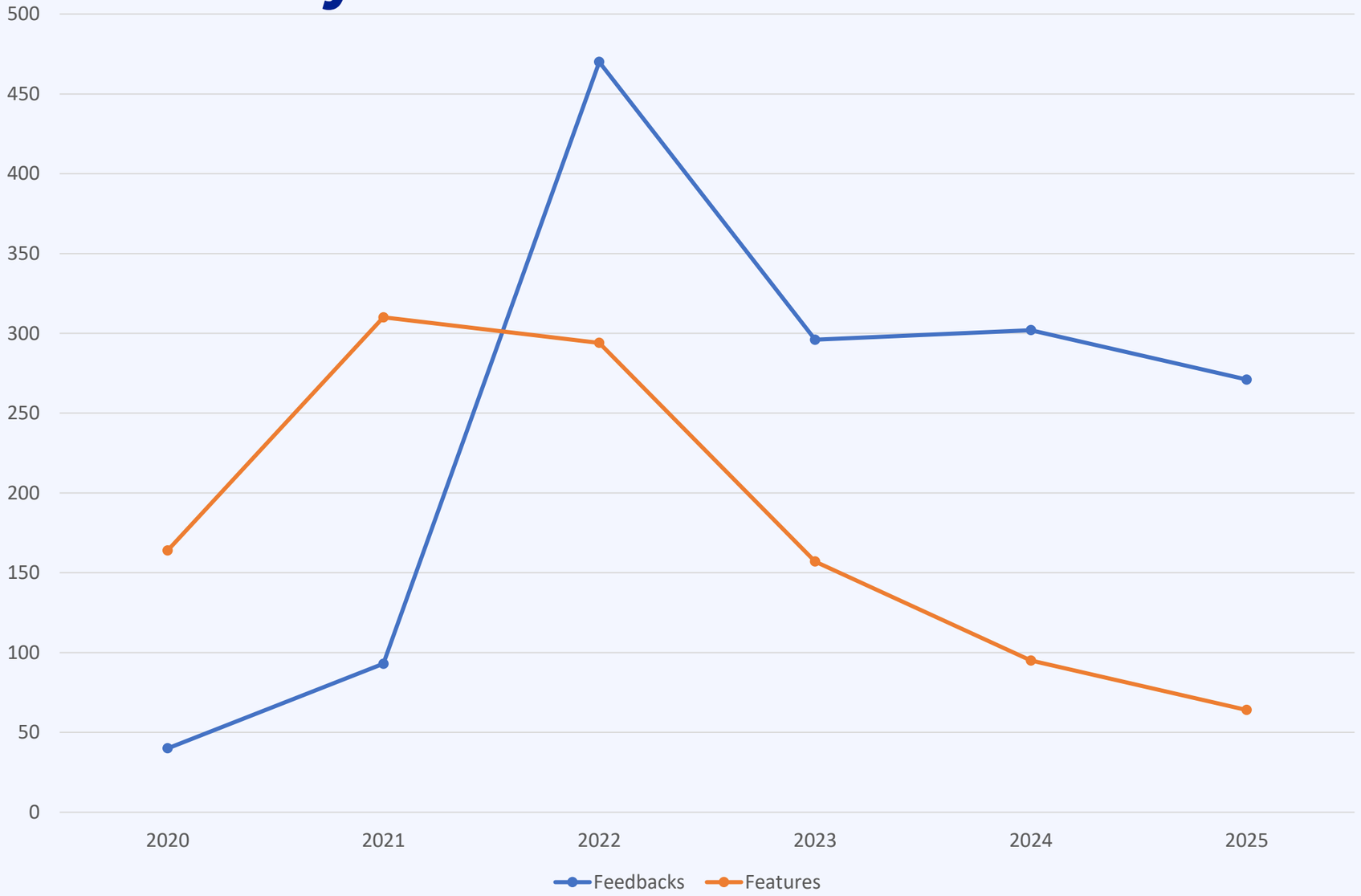
Maintenance

What needs updating?

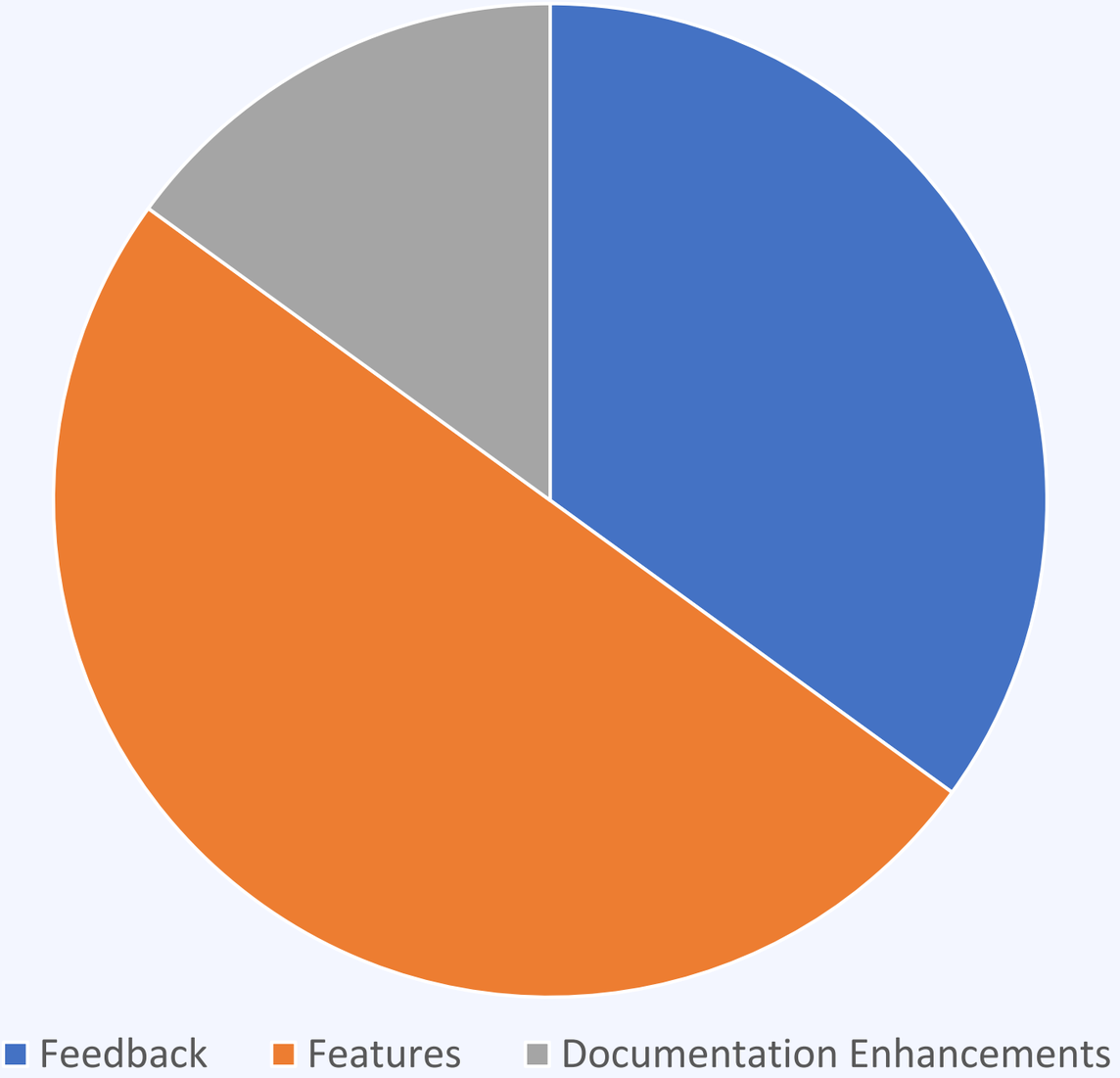
Priority

What should be addressed?

The Reality at Scale



Time Spent



What the Numbers Don't Show

Context switching

Every feedback item pulls you out of flow

Investigation

Understanding the real issue takes time

Coordination

Multiple stakeholders, async communication



Why Did There Seem So Much Feedback?

1. **Product changes**
   Features evolve, UI updates, flows shift
2. **Edge cases emerge**
   Real usage reveals scenarios not documented
3. **Documentation decays**
   Without updates, docs become unreliable
4. **It should have been reviewed**
   Users find what reviewers miss

The Small Stuff Counts

1

Typos

Erode trust in accuracy

2

Phrasing

Ambiguity creates confusion

3

Completeness

One missing step breaks everything





CHAPTER 2

Reading the Feedback

Useful Feedback

> "I followed step 3 exactly as written, but the API returns a 403 error. Is there a permission I'm missing? The error message doesn't match what's described in the troubleshooting section."

Real feedback is specific, contextual, and often reveals gaps in our assumptions

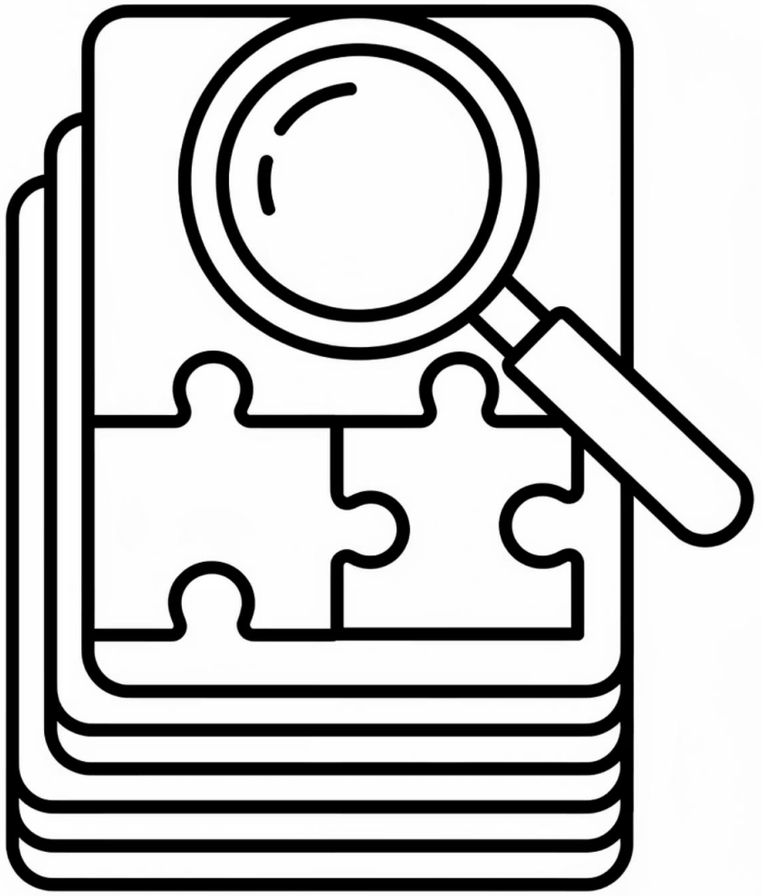




Less Useful Feedback

> "The documentation doesn't work."

Vague feedback lacks context: Which topic? What were you trying to do? What happened instead? Without specifics, we can't identify or fix the actual issue.



When Feedback Is Vague, Ask Better Questions

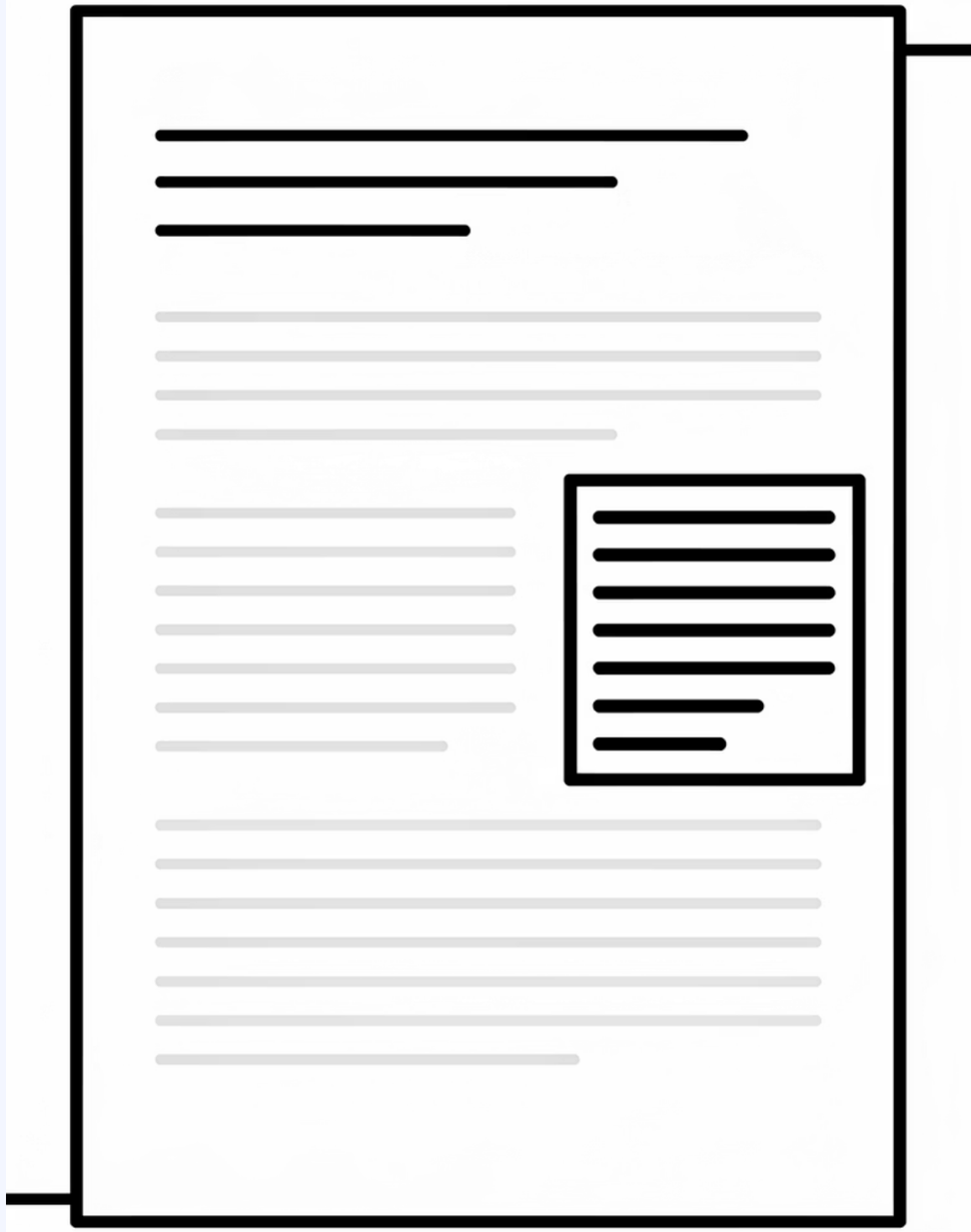
- Which topic is affected?
- Which step or section?
- What is incorrect or missing?
- What did the user misunderstand?
- Why did they misunderstand?
- Does this affect other areas of the knowledge center?

Answers Determine the Fix

Insights from feedback analysis lead to specific actions

- Add a missing step
- Convert a note into a warning
- Clarify or rephrase instructions
- Search the knowledge center for related or impacted topics

Precision and scope control, not large-scale rewrites



CHAPTER 3

Who Gives the Feedback?

Understanding your sources shapes how you respond

Product Managers

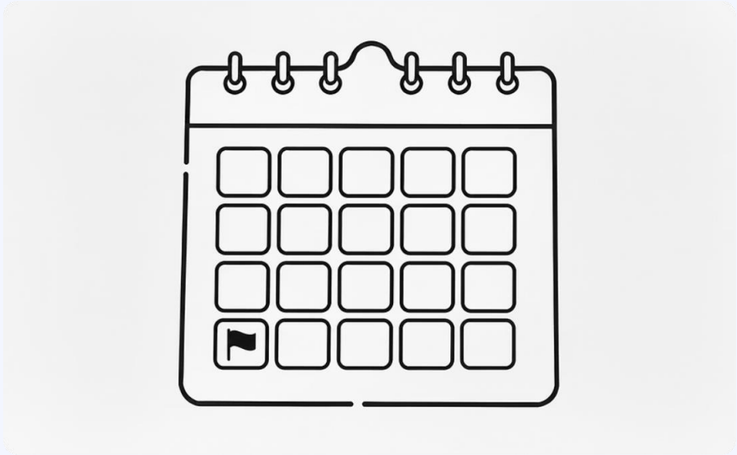
Support

R&D

End-users



Product Manager Feedback



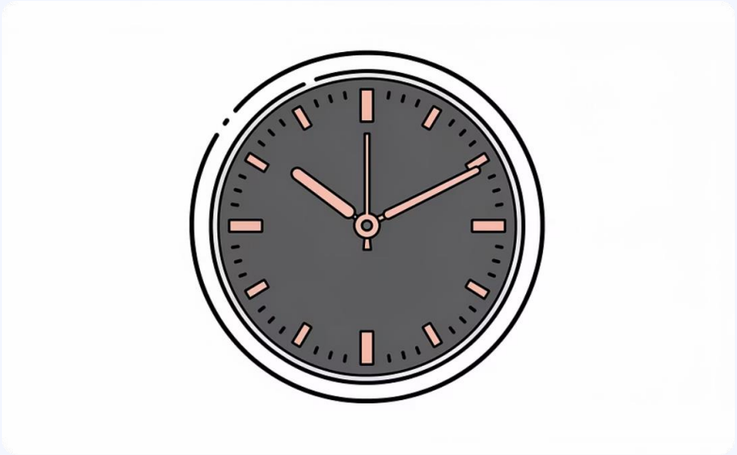
Intent

Aligning docs with product vision



Message Accuracy

Catching messaging issues early



Timing

Frequently arrives too late

Support as a Feedback Channel



Urgency

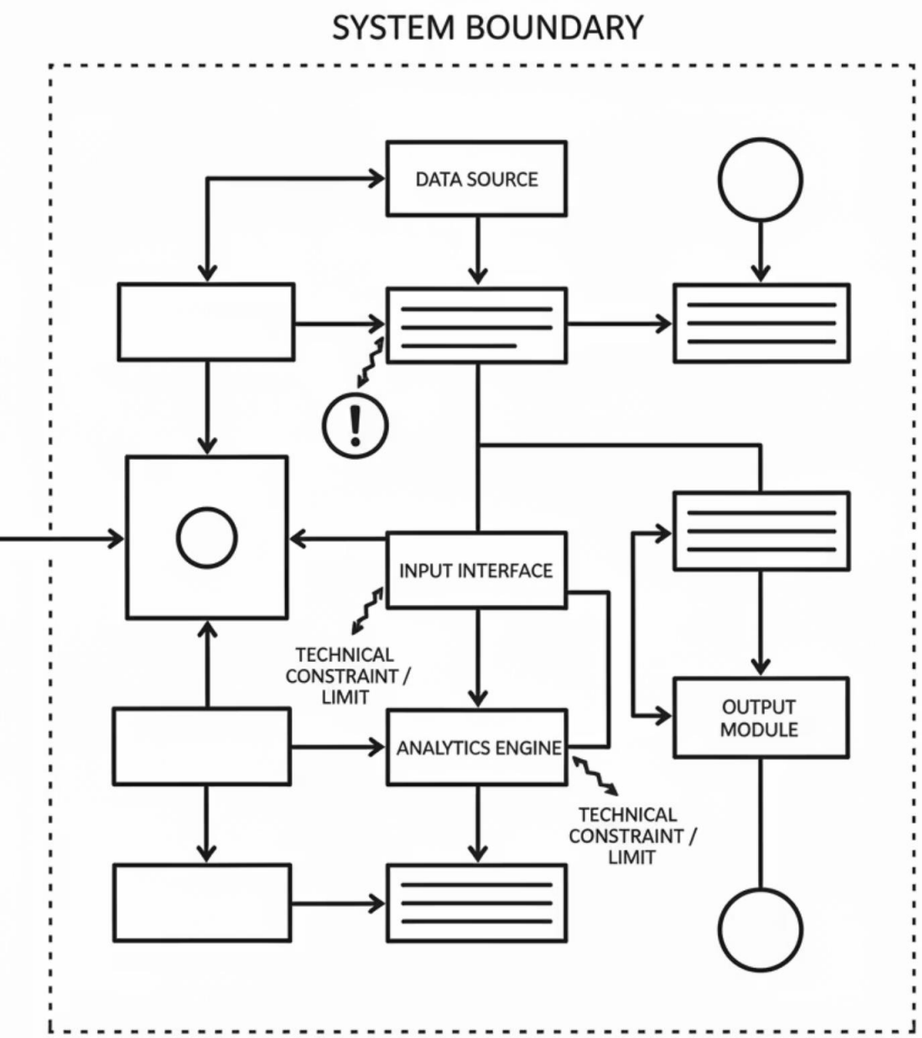
Users blocked right now

Context

Real error messages, specific environments

Aggregation

Patterns emerge from ticket volume



R&D Feedback

Technical Accuracy

R&D help catch gaps between design and implementation

Edge Cases

Critical constraints and operational limitations. Not all edge-cases require documentation

Maintenance

Bugs and “improvements” require updates from R&D

End-User Feedback



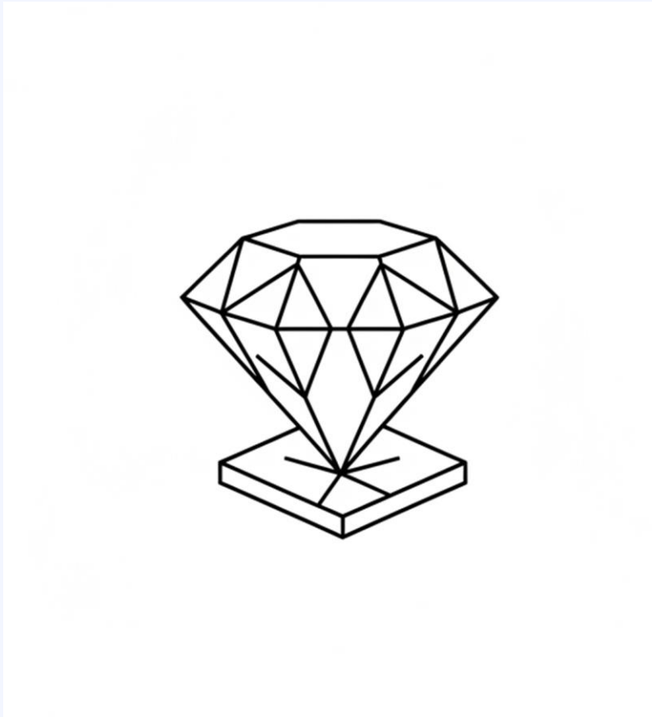
High value

Shows real-world usage



High noise

Requires filtering and interpretation

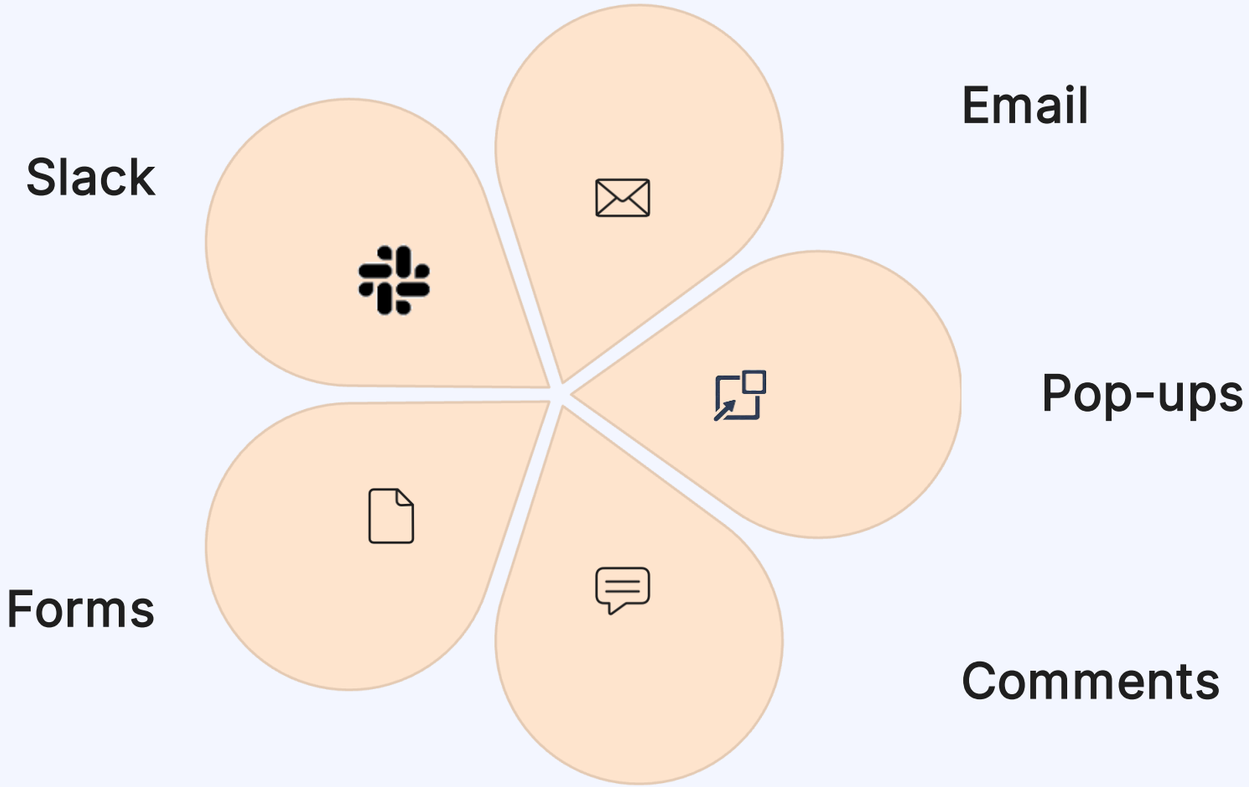


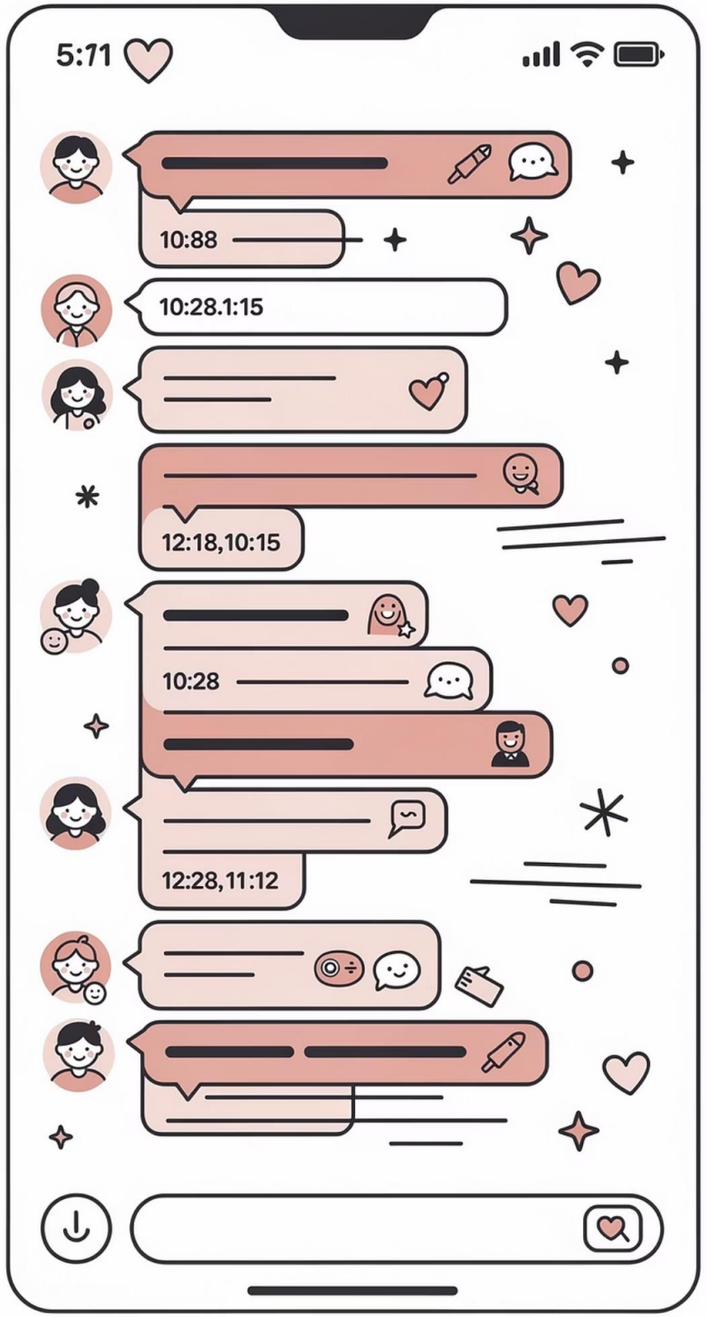
Rare

Customers do not communicate directly with TWs.

CHAPTER 4

Feedback Channels





Unstructured Feedback

Slack & Email

Fast

Low friction, immediate response

Unstructured

Hard to track or prioritize

Fragile

Lost in threads, buried in archives

Tufin solution:

- Convert discussions into Jira feedbacks
- Use iconography to track status of discussions

When Docs Become a Discussion

Additional advantages/disadvantages



Engagement

Users invested enough to engage



Easy conversion

Comments become official doc feedback requests



Moderation

Requires active monitoring to keep discussions focused



Inline Feedback

Pop-up widgets on doc pages

Low friction

Captured in context

Timing matters

Users give feedback when frustrated

Strong signal

Page-level metrics reveal problem areas

Structured Feedback

Jira Doc Feedback with Required Fields

Quality

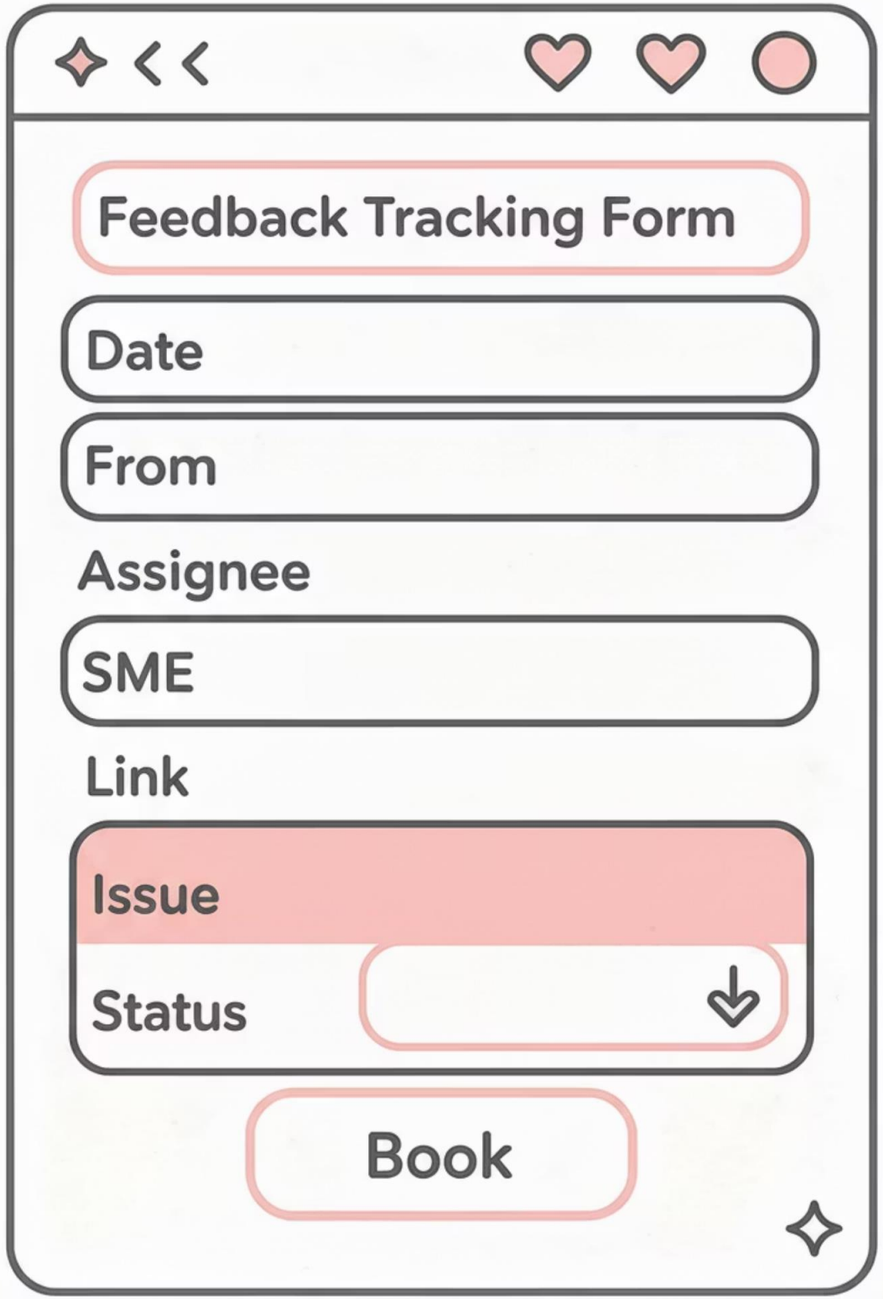
Fields enforce useful information

Friction

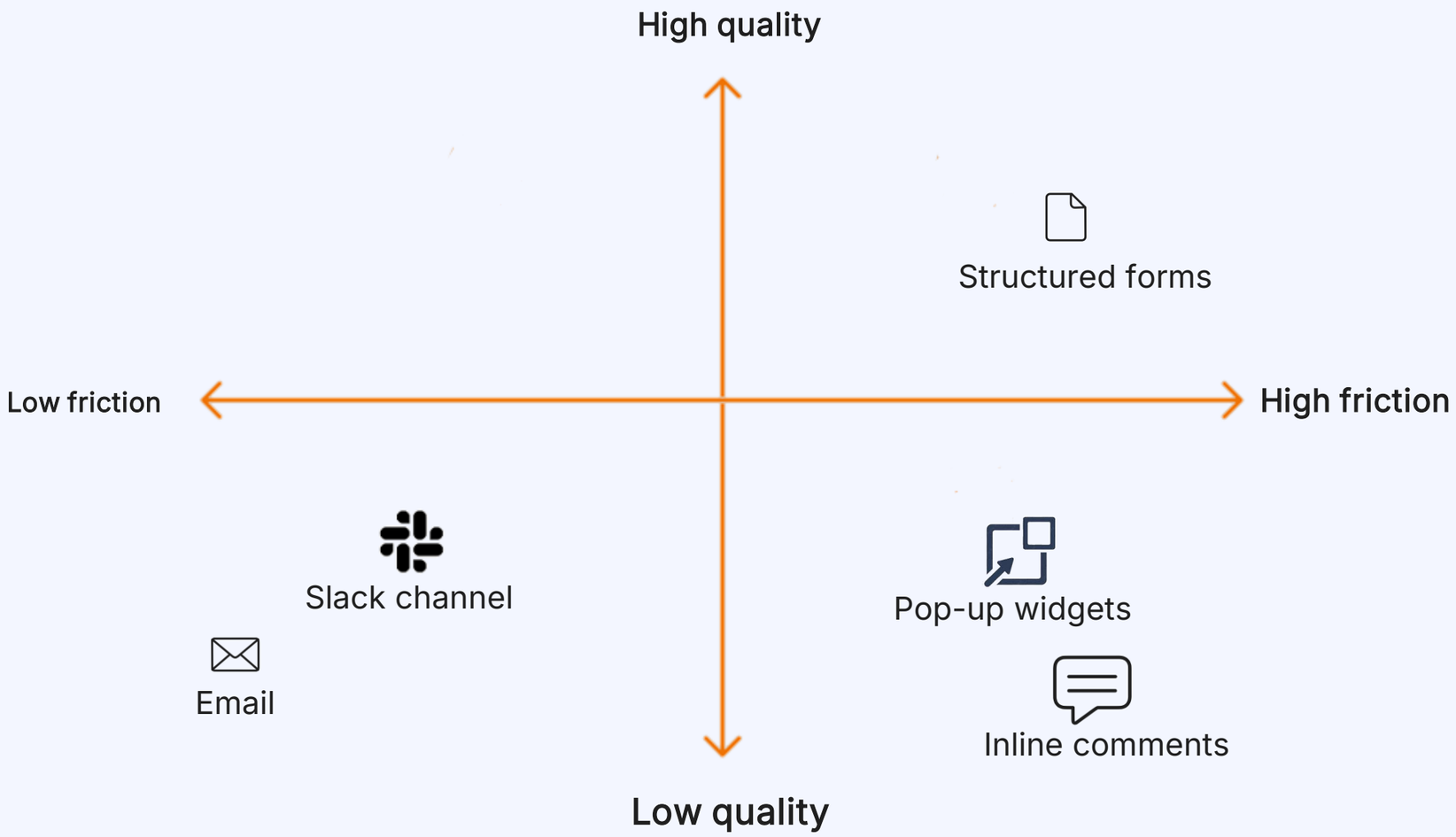
Reduces volume, increases value

Intent

Users commit to describing issues



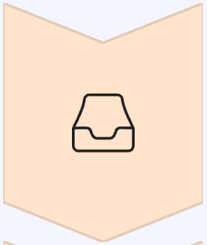
Designing Feedback Channels



Every channel design is a deliberate tradeoff between ease of submission and quality of information

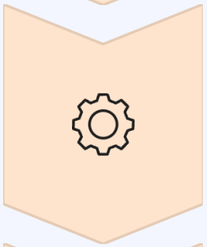
CHAPTER 5

Managing Feedback



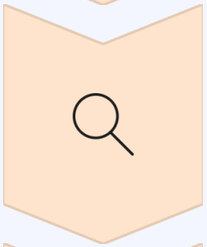
To Do

Feedback received and logged



In Progress

Actively being addressed



In Review

Documentation waiting for approval



Accepted

Documentation published. Feedback provider notified

Visibility Enables Speed

Ownership

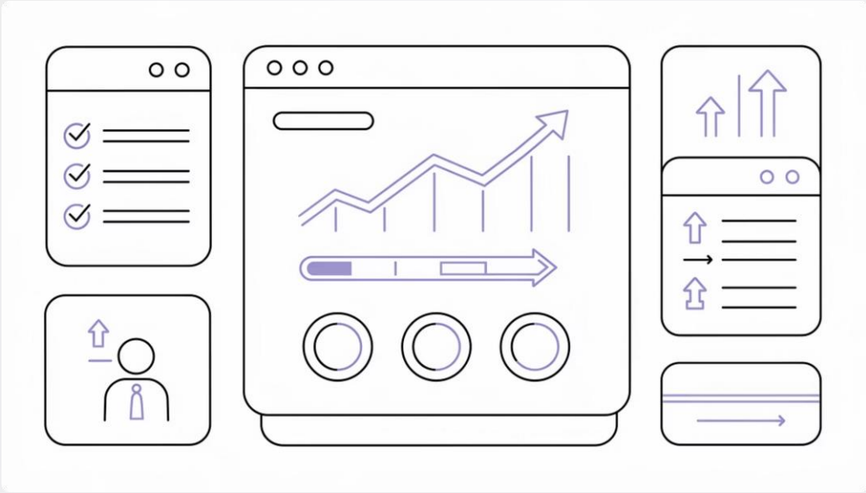
Everyone knows who's responsible

Clarity

Status is transparent

Momentum

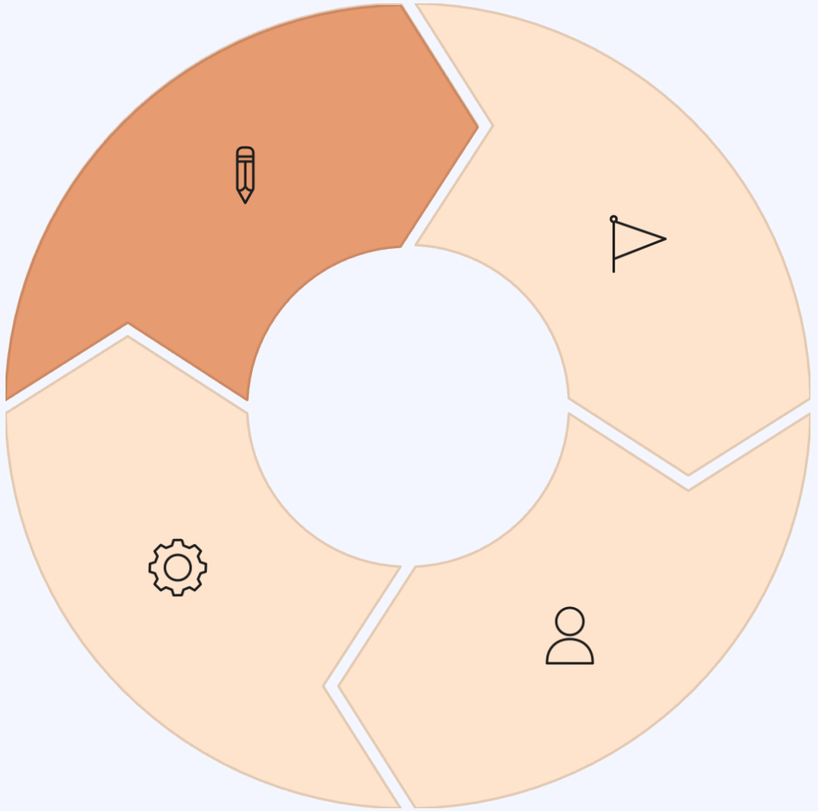
Nothing sits unnoticed



Who Decides

Docs team

Clarity, accuracy, scope



Product

Strategy, roadmap fit

R&D

Technical accuracy

Requester

Does it solve their problem?

Approval Holdups

Bottlenecks

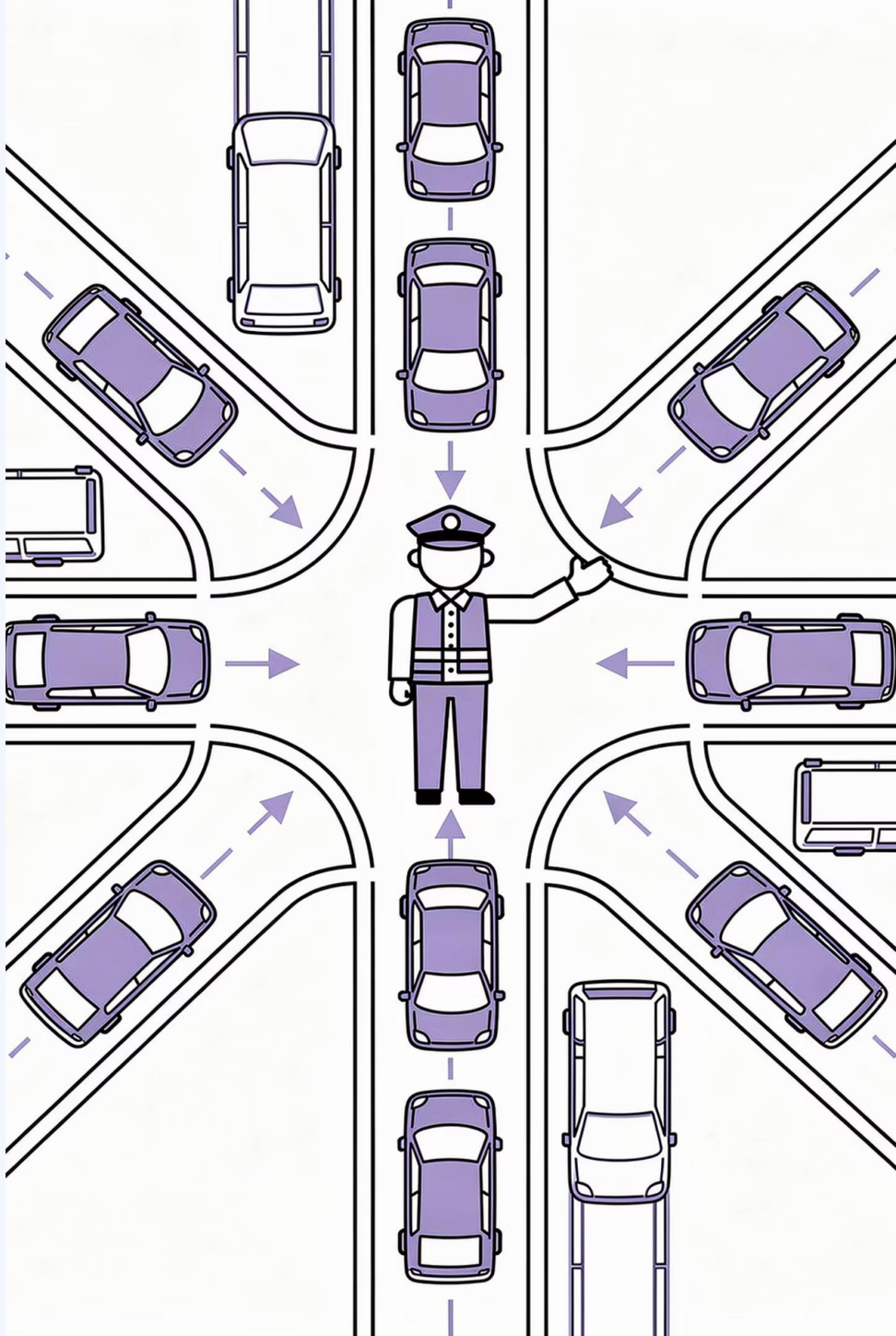
One person gates everything

Late input

Reviewers appear after work is done

Analysis paralysis

Endless discussion, no decision





When No Is the Right Answer

> Not all feedback should be implemented

Scope

Request falls outside documentation purpose

Clarity

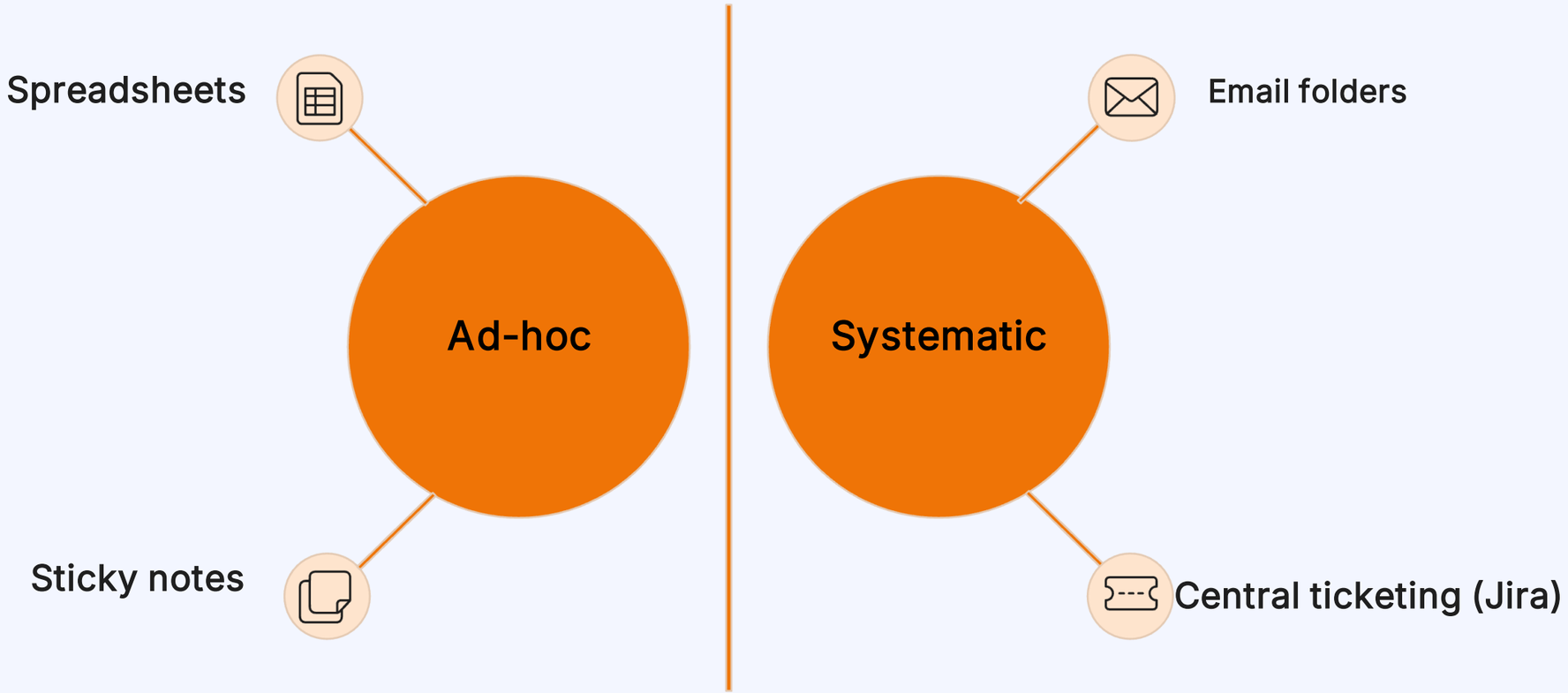
Explaining why builds trust

Trust

Saying no thoughtfully preserves relationships

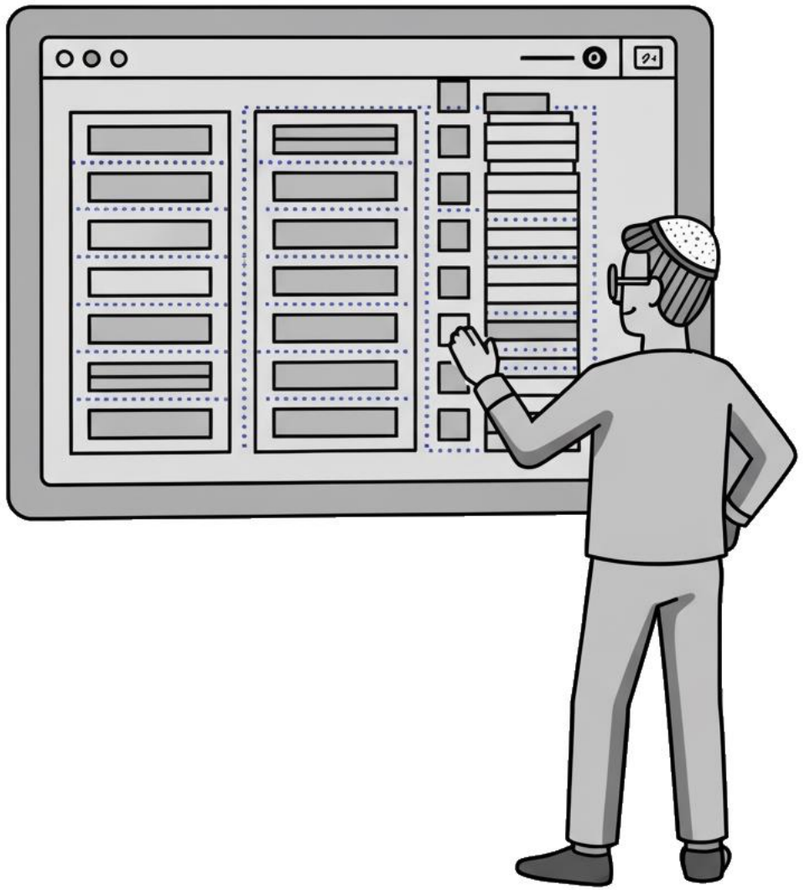
CHAPTER 6

Tracking Feedback



Ad hoc tracking breaks down at scale

Using Jira



- **Visibility**

  Everyone sees the board

- **Scalability**

  Can handle thousands of feedback items

- **Overhead**

  Requires maintenance



Planned Work vs Feedback

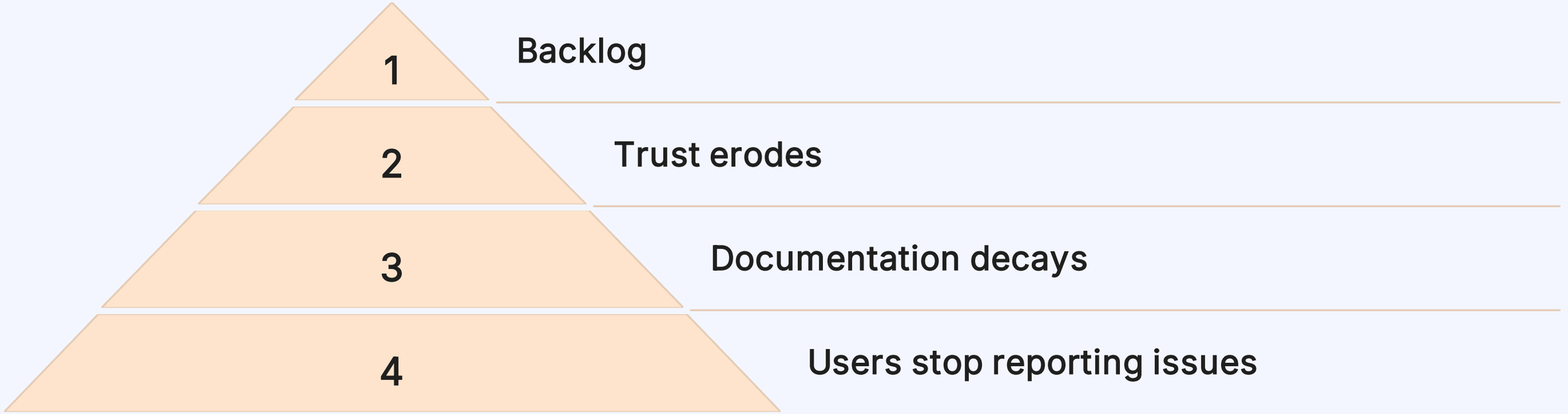
Interruption

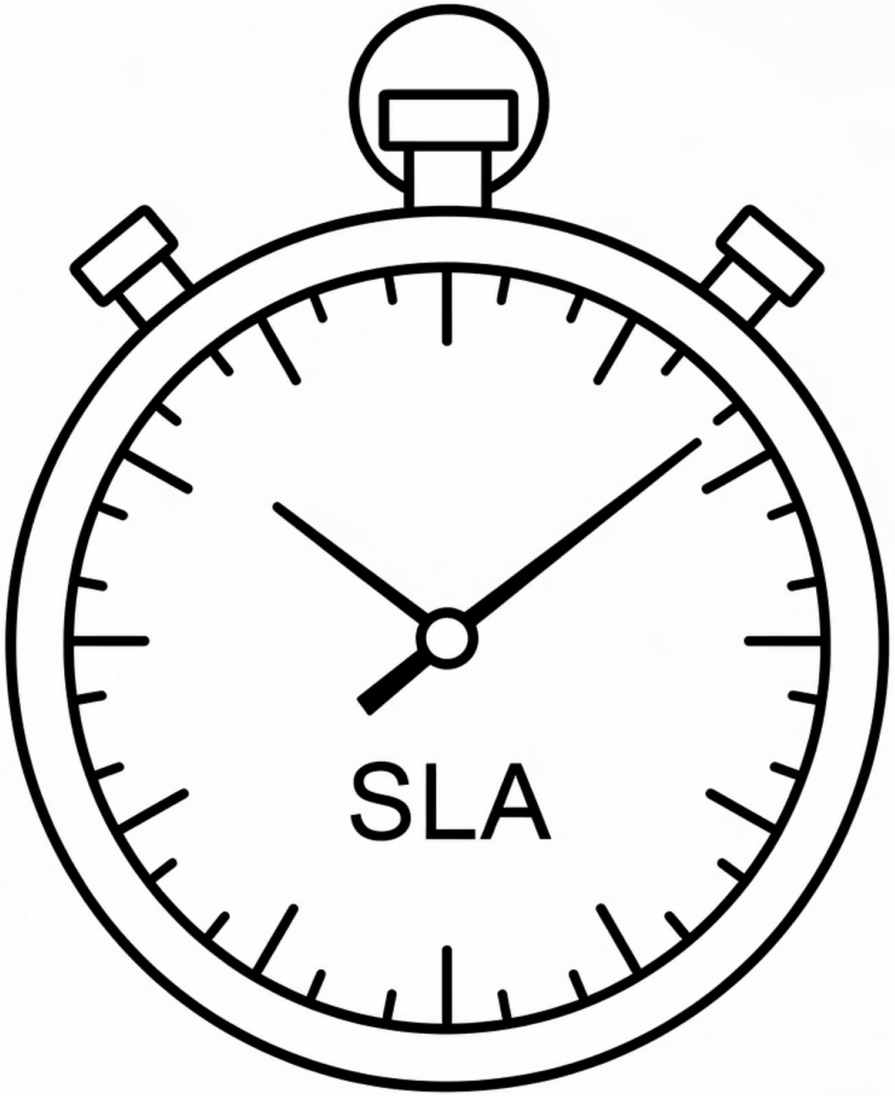
Feedback disrupts scheduled work

Tradeoff

Every feedback fix delays new content

Feedback Debt





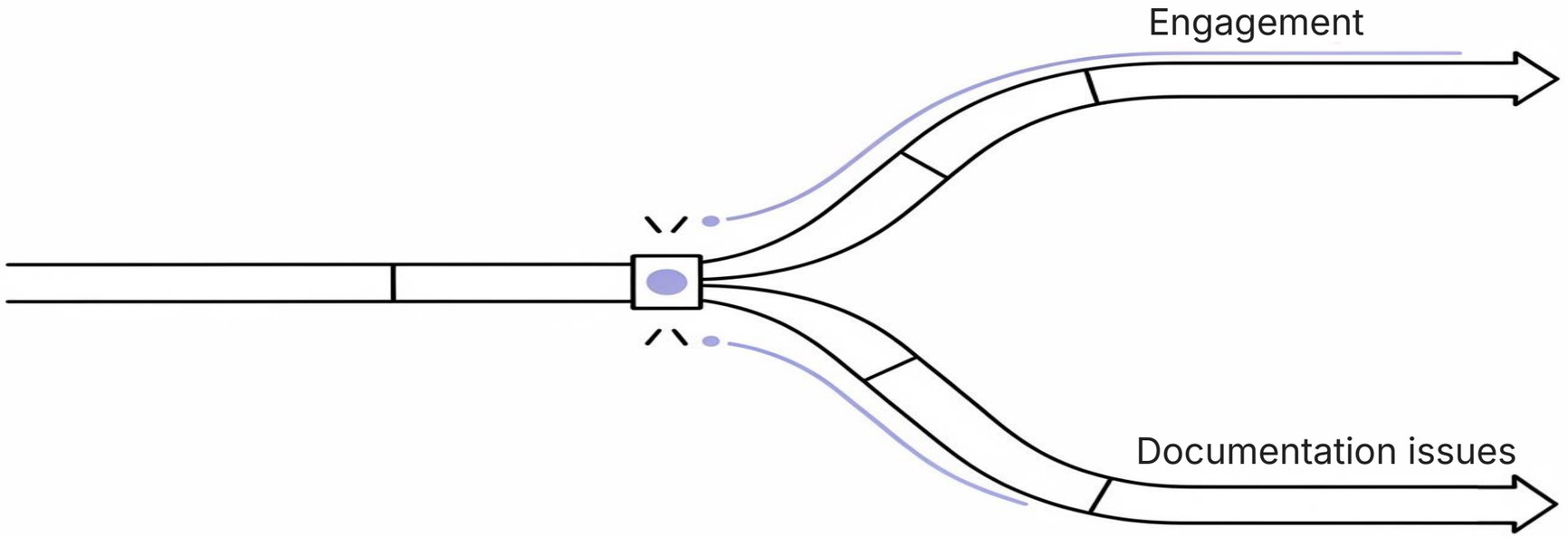
Should Docs Have SLAs?

Arguments for

- Sets clear expectations
- Creates accountability
- Prevents feedback from languishing

Arguments against

- Every issue is different
- Complexity varies wildly
- Gaming the metrics



More Feedback Means Better Docs?

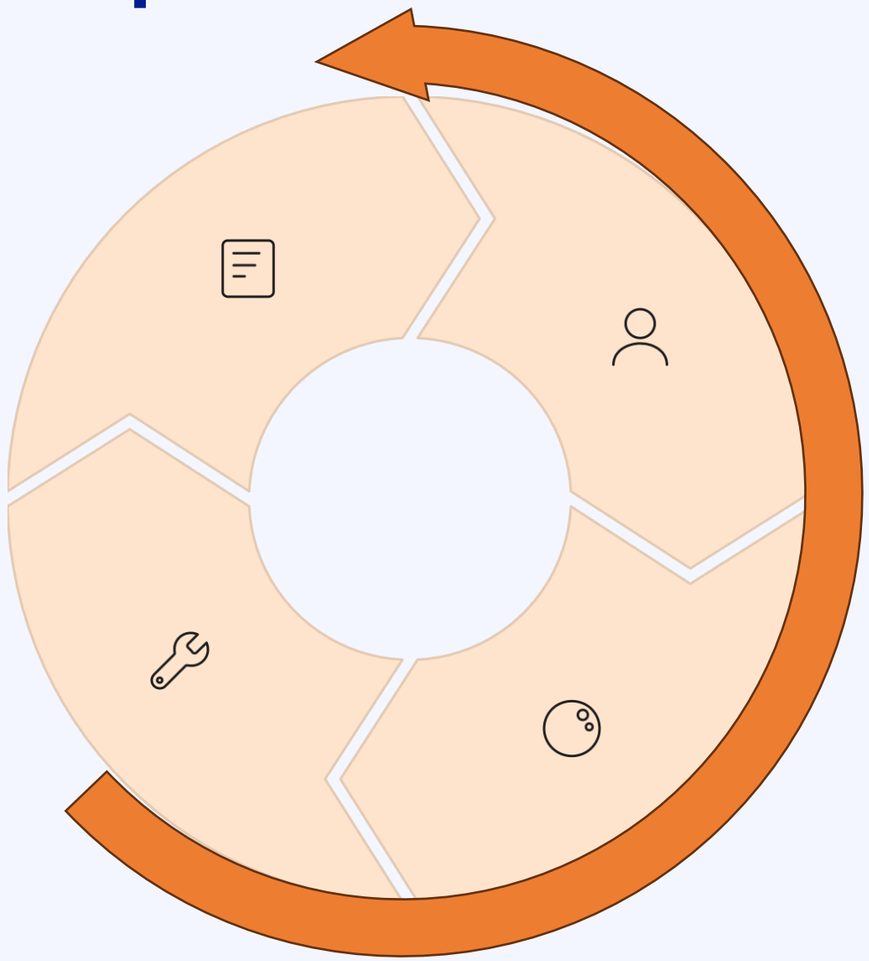
> Is increasing feedback volume a success signal or a warning sign?

High feedback volume might mean engagement — or it might mean your docs aren't working

Feedback Is a Loop

Publish

Documentation goes live



Users read

Real-world usage begins

Feedback flows

Users respond with questions and issues

Improve

Writers update and refine

Bonus Chapter!

How AI Can Help

Triage

Classify feedback by type and urgency

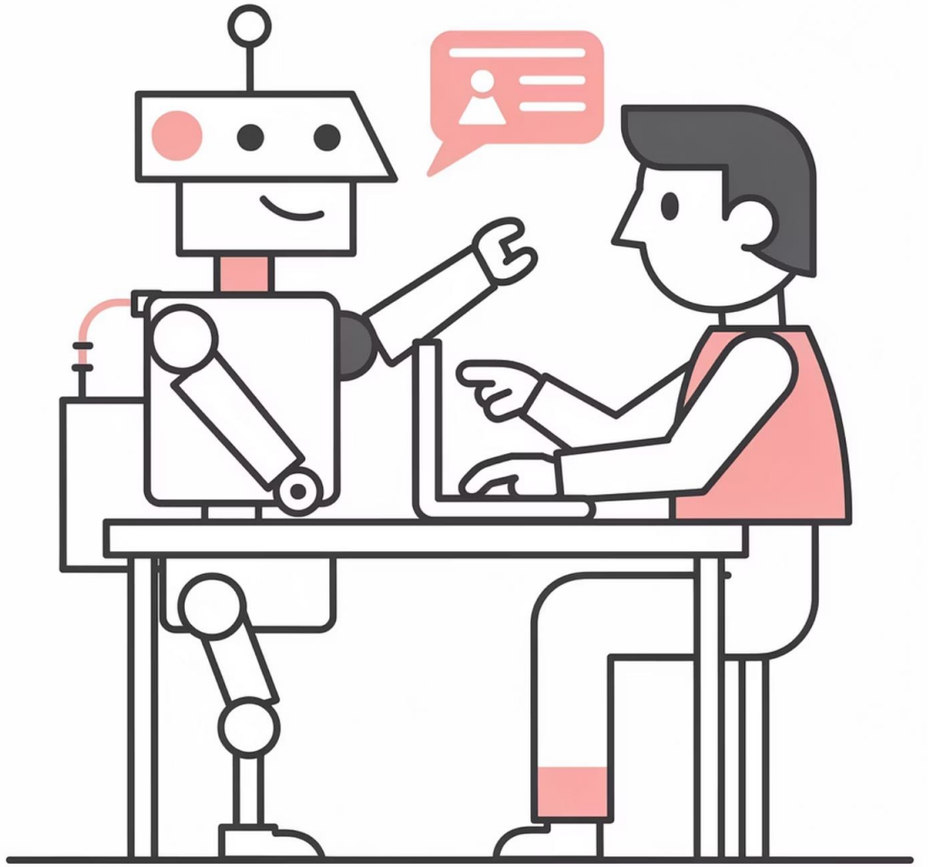
Tagging

Automated categorization at scale

Draft responses

Suggest fixes, not final answers

AI assists, humans decide



AI Personas as Reviewers

Simulate different reader perspectives before publishing

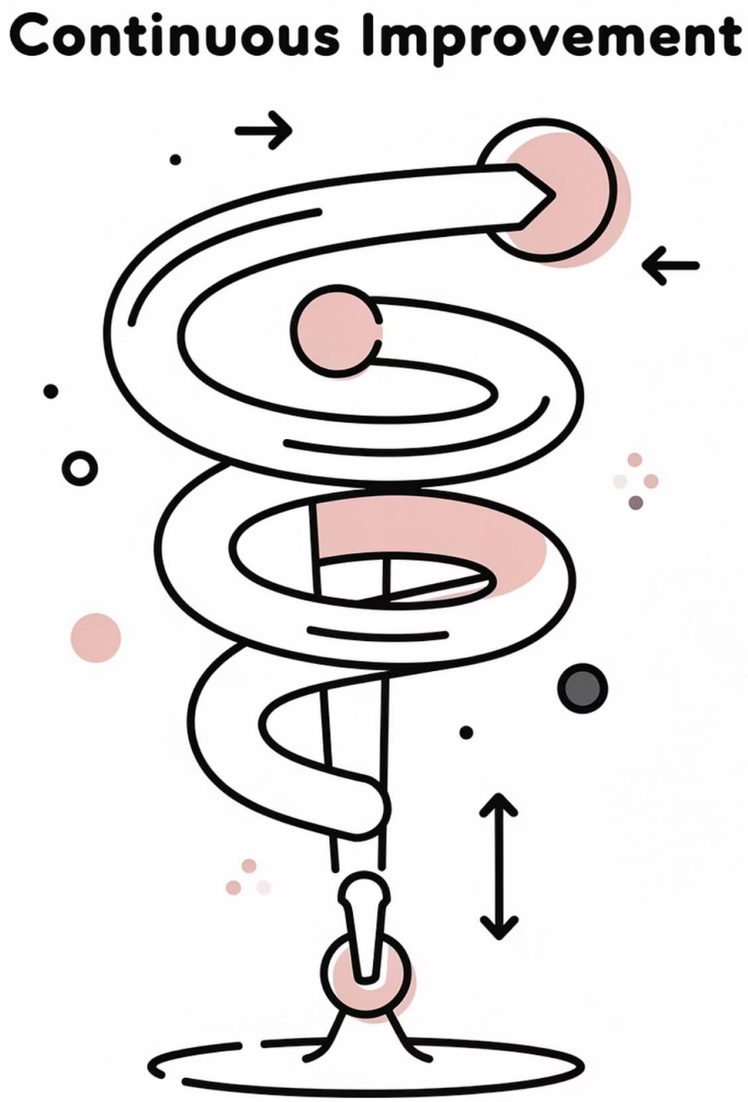
Catch assumptions, ambiguities, confusing instructions

Anything that will make the user fail



Blind spots

AI reveals what writers assume readers know



RTFM 2.0 Takeaways

Listen

Feedback is signal, not noise

Track

Systems beat ad hoc responses

Decide

Not all feedback gets implemented

Iterate

Documentation is never finished

Thank You

Any Questions?

Email: yehoshua.paul@tufin.com

